Verschlüsselungskonzept — Shinpai-AI

Das Verschlüsselungskonzept von Shinpai-AI — Komplett erklärt

- Inhaltsverzeichnis
- 1. Worum geht's überhaupt?
- 2. Abkürzungsverzeichnis — jeder Buchstabe erklärt
- 3. Die drei Feinde, gegen die verschlüsselt wird
- 4. Kryptographie-Grundlagen in Alltagssprache
- 5. Post-Quantum — warum das nicht nur Buzzword ist
- 6. Die Bausteine im Detail
- 7. Die Schlüssel-Kaskade (KEK → DEK → Daten)
- 8. Der Salzstreuer — was er macht und warum
- 9. Zwei-Faktor-Authentifizierung (2FA)
- 10. Die drei Sicherheits-Schichten
- 11. Recovery: Seed-Phrase und Passwort-Vergessen
- 12. Was Shinpai-AI **nicht** tut — die Eisernen Regeln
- Anhang: Wo welche Komponente im Code lebt

Das Verschlüsselungskonzept von Shinpai-AI — Komplett erklärt

> ***Zweck dieser Doku:*** *Jeder Begriff, jede Abkürzung, jeder Schritt — so erklärt, dass man es verstehen kann* ***ohne*** *Krypto-Studium. Gleichzeitig so genau, dass man es einem Security-Experten vorzeigen kann ohne sich zu schämen.*
>
> ***Stand:*** *2026-04-22* ***Für:*** *Kneipe, ShinNexus, ShinShare, Shidow* ***Basis:*** `/Doku/Programm-Entwicklung/PQ-Architektur.md`

Inhaltsverzeichnis

1. Worum geht's überhaupt?
2. Das Abkürzungsverzeichnis — jeder Buchstabe erklärt
3. Die drei Feinde, gegen die verschlüsselt wird
4. Kryptographie-Grundlagen in Alltagssprache
5. Post-Quantum — warum das nicht nur Buzzword ist
6. Die Bausteine im Detail
7. Die Schlüssel-Kaskade (KEK → DEK → Daten)
8. Der Salzstreuer — was er macht und warum
9. Zwei-Faktor-Authentifizierung (2FA)
10. Die drei Sicherheits-Schichten
11. Recovery: Seed-Phrase und Passwort-Vergessen
12. Was Shinpai-AI **nicht** tut — die Eisernen Regeln

---

1. Worum geht's überhaupt?

Ein Server speichert **Daten**. Benutzernamen, Passwörter, Chatnachrichten, Profilbilder, Tisch-Zugehörigkeiten. Sobald diese Daten auf einer Festplatte liegen, sind sie prinzipiell lesbar — **außer** man verschlüsselt sie.

Verschlüsseln heißt: Die Daten werden mit einem mathematischen Verfahren so durcheinander gewürfelt, dass sie aussehen wie Zufallszahlen. Nur wer den **Schlüssel** hat, kann die Daten wieder zurückwürfeln (entschlüsseln).

Bei Shinpai-AI ist die Grundregel einfach:

> ***Die Server sind blinde Postboten.*** *Sie transportieren verschlüsselte Päckchen. Sie sehen nicht, was drin ist.*

Das hat drei Konsequenzen:

1. **Wer die Festplatte klaut, hat Datenmüll.** Ohne Schlüssel sind die Bytes wertlos.
2. **Wer den Server hackt, sieht nur Verschlüsseltes.** Die Angreifer müssen jedes Schloss einzeln knacken.
3. **Selbst der Betreiber (Owner) kann nicht alles lesen.** Persönliche User-Daten bleiben auch vor ihm verschlossen.

Die gesamte folgende Komplexität dient **diesem einen Ziel**. Jede technische Entscheidung wird gemessen an: *"Gibt das dem Angreifer einen Weg rein?"* Wenn ja → nicht machen.

---

2. Abkürzungsverzeichnis — jeder Buchstabe erklärt

Kryptographie-Grundbegriffe

Abkürzung	Ausgeschrieben	Was es bedeutet
PQ	**P**ost-**Q**uantum	“Nach-Quanten-sicher” — funktioniert auch dann noch, wenn Quantencomputer gebaut werden.
KDF	**K**ey **D**erivation **F**unction	“Schlüssel-Ableitungs-Funktion” — macht aus einem Passwort einen echten Verschlüsselungs-Schlüssel.
MAC	**M**essage **A**uthentication **C**ode	“Nachrichten-Echtheits-Code” — beweist, dass eine Nachricht nicht verändert wurde.
AEAD	**A**uthenticated **E**ncryption with **A**ssociated **D**ata	“Authentifizierte Verschlüsselung” — verschlüsselt **und** stellt Echtheit sicher, alles in einem.
IV / Nonce	**I**nitialization **V**ector / “Number used once”	Eine Einmal-Zufallszahl die mit jedem Verschlüsselungs-Vorgang mitgegeben wird, damit zweimal derselbe Klartext nie identisch verschlüsselt aussieht.

Die Schlüssel

Abkürzung	Ausgeschrieben	Was es bedeutet
KEK	**K**ey **E**ncryption **K**ey	“Schlüssel-zum-Verschlüsseln-anderer-Schlüssel”. Der oberste, aus dem Passwort abgeleitete Key. Verschlüsselt nur weitere Schlüssel, nie direkt Daten.
DEK	**D**ata **E**ncryption **K**ey	“Daten-Verschlüsselungs-Schlüssel”. Der echte Arbeits-Schlüssel, mit dem die tatsächlichen Nutzer-Daten ver- und entschlüsselt werden.

Algorithmen

Abkürzung	Ausgeschrieben	Was es bedeutet
AES	**A**dvanced **E**ncryption **S**tandard	Der Welt-Standard für symmetrische Verschlüsselung seit 2001 (AES-256 mit 256-bit Schlüssel ist militärischer Standard).
GCM	**G**alois/**C**ounter **M**ode	Betriebsart von AES, die nicht nur verschlüsselt, sondern auch gleich die Echtheit prüft (AEAD-fähig).
AES-256-GCM	= AES mit 256-bit Schlüssel im GCM-Modus	Unsere Standard-Symmetrik. Schnell, sicher, manipulations-sicher.
SHA	**S**ecure **H**ash **A**lgorithm	"Sichere Quersummen-Berechnung". Macht aus beliebigen Daten einen festen Fingerabdruck.
SHA-256	= SHA mit 256-bit Ausgabe	Ein 64-Zeichen langer Fingerabdruck. Einbahnstraße — vom Fingerabdruck kommt man nie zurück auf die Originaldaten.
Argon2	(Kein Akronym — Eigenname)	Der Gewinner der Password Hashing Competition 2015. Extra **langsam und speicherhungrig** designed, um Passwort-Knacker auszubremsen.
Argon2id	Argon2 **id**entity-Variante	Hybrid aus Argon2**i** (Seiten-Kanal-sicher) + Argon2**d** (GPU-resistent). Die empfohlene Standard-Variante.
ML-KEM	**M**odule-**L**attice-based **K**ey **E**ncapsulation **M**echanism	"Gitter-basierter Schlüssel-Einkapselungs-Mechanismus". Ein Verfahren, mit dem zwei Parteien über einen öffentlichen Kanal einen geheimen Schlüssel austauschen können. PQ-sicher.
ML-KEM-768	ML-KEM in der "768-Bit-Härte"-Variante	Sicherheits-Level ≈ AES-192. Der Standard-Kompromiss aus Geschwindigkeit und Sicherheit (NIST-Empfehlung für die meisten Anwendungen).

Abkürzung	Ausgeschrieben	Was es bedeutet
ML-DSA	**M**odule-**L**attice-based **D**igital **S**ignature **A**lgorithm	"Gitter-basierter digitaler Signatur-Algorithmus". Verfahren um Dokumente unfälschbar zu unterschreiben. PQ-sicher.
ML-DSA-65	ML-DSA in der "65"-Sicherheits-Variante	Sicherheits-Level ≈ AES-192. Unsere Standard-Signatur.
Kyber	(Älterer Name für ML-KEM)	Projektname vor der NIST-Standardisierung. Ist exakt dasselbe wie ML-KEM-768.
Dilithium	(Älterer Name für ML-DSA)	Projektname vor der NIST-Standardisierung. Ist exakt dasselbe wie ML-DSA-65.

Authentifizierung

Abkürzung	Ausgeschrieben	Was es bedeutet
2FA	**2**-**F**aktor-**A**uthentifizierung	Zwei unabhängige Beweise der Identität: etwas das du **weißt** (Passwort) + etwas das du **hast** (z.B. dein Handy mit Authenticator-App).
TOTP	**T**ime-based **O**ne-**T**ime **P**assword	"Zeit-basiertes Einmal-Passwort". Der 6-stellige Code, der alle 30 Sekunden in der Authenticator-App wechselt.
OTP	**O**ne-**T**ime **P**assword	"Einmal-Passwort". Oberbegriff. TOTP ist die zeit-basierte Unterart.
HOTP	**H**MAC-based **O**ne-**T**ime **P**assword	Die zähler-basierte Variante (wechselt bei Knopfdruck, nicht alle 30 Sek). Nutzen wir nicht.
HMAC	**H**ash-based **M**essage **A**uthentication **C**ode	Spezielle MAC-Art, die einen Hash-Algorithmus nutzt. Baustein für TOTP.

Standardisierung

Abkürzung	Ausgeschrieben	Was es bedeutet
NIST	**N**ational **I**nstitute of **S**tandards and **T**echnology	US-Behörde, die Krypto-Standards festlegt. Wenn NIST etwas zertifiziert, wird es weltweit genutzt.
FIPS	**F**ederal **I**nformation **P**rocessing **S**tandards	Die Standards selbst. **FIPS-203** = ML-KEM. **FIPS-204** = ML-DSA. **FIPS-197** = AES. Das sind offizielle Nummern.

Identität und Session

Abkürzung	Ausgeschrieben	Was es bedeutet
API	**A**pplication **P**rogramming **I**nterface	“Programmierschnittstelle”. Die Türen, durch die Programme miteinander reden.
Token	(kein Akronym, direkt aus dem Englischen)	“Marke” / “Chip”. Ein zufälliger String, der als Eintrittsbeleg dient. Wie ein Kino-Bon: kurzlebig, ersetzbar, beweist Zugangsrecht.
Bearer-Token	“Inhaber-Token”	Art von Token, bei dem **wer ihn hat, ihn nutzen darf** — wie Bargeld. Deshalb muss er über TLS/HTTPS laufen, sonst klaubar.
Session	“Sitzung”	Der Zeitraum zwischen Login und Logout (oder Auto-Abmeldung). Gültigkeit des Tokens.
Seed-Phrase	“Saatgut-Wortreihe”	Die 12 oder 24 Wörter, aus denen kryptographische Schlüssel deterministisch wieder herstellbar sind (wie bei Bitcoin-Wallets).
Vault	“Tresor”	Ein lokal verschlüsselter Datenspeicher. Bei uns: der Server-lokale, mit KEK-Kaskade geschützte Daten-Ordner.
Igni	(Eigenname, lat. “ignis” = Feuer, hier “Zündschlüssel”)	Die optionale Datei, die nach Owner-Login den Master-Key maschinen-gebunden zwischenspeichert. Wie ein Auto-Funkschlüssel.

Systemumgebung

Abkürzung	Ausgeschrieben	Was es bedeutet
RAM	**R**andom **A**ccess **M**emory	Arbeitsspeicher. Daten, die hier liegen, sind weg sobald der Strom ausgeht. Wichtig: **Schlüssel leben hier**, nicht auf Platte.
machine-id	—	Ein eindeutiger, von Linux/Windows vergebener Computer-Fingerabdruck. Existiert pro Betriebssystem-Installation genau einmal.
CFFI	**C** **F**oreign **F**unction **I**nterface	Mechanismus, mit dem Python auf C-Bibliotheken zugreift. Wir nutzen `argon2-cffi` um die C-Referenz-Implementation von Argon2 einzubinden.

---

3. Die drei Feinde, gegen die verschlüsselt wird

Verschlüsselung ist immer gegen **konkrete Bedrohungs-Modelle** gebaut. Unsere drei Gegner:

Gegner 1: Der Festplatten-Dieb

Szenario: Server wird geklaut. Gehäuse aufgeschraubt. Festplatte ausgebaut. An anderen Computer angeschlossen. Jemand versucht, die Dateien zu lesen.

Abwehr: Wenn der Server **aus** ist, sind alle Schlüssel weg (sie waren nur im RAM). Die Dateien auf der Platte sind mit AES-256-GCM verschlüsselt. Ohne den Schlüssel nicht lesbar — selbst mit allen Quantencomputern der nächsten 100 Jahre nicht.

Gegner 2: Der Online-Hacker

Szenario: Der Server läuft. Angreifer findet eine Sicherheitslücke in der Software (SQL-Injection, Exploit). Bekommt Zugriff auf die Datenbank.

Abwehr: Die Datenbank enthält nicht die **Daten**, sondern nur **verschlüsselte Blobs**. Selbst voller DB-Zugriff gibt nicht den Inhalt der Chats, Profile, privaten Nachrichten. Der Angreifer müsste zusätzlich: - Den DEK im RAM abgreifen (extrem schwer, erfordert Kernel-Lücke) - **Jeden User-Datenschlüssel separat** knacken (unmöglich ohne dessen Passwort)

Gegner 3: Der Owner selbst (!)

Szenario: Der Server-Betreiber will heimlich die Privatnachrichten seiner User lesen. Oder: Eine Behörde zwingt ihn, sie herauszugeben.

Abwehr: Der Owner hat **Schicht 1 auf**, kann also die Datenbank-Struktur sehen (wer ist wann online, wer postet in welchem Tisch). Aber die **Inhalte** der User-Blobs sind mit **User-Datenschlüsseln** verschlüsselt, und die stehen dem Owner **nicht** zur Verfügung. Sie werden im Browser des Users aus dessen Passwort abgeleitet — das Passwort selbst bekommt der Server nie zu sehen.

Das nennt man **Zero-Knowledge-Architektur**: Der Dienst-Anbieter weiß nichts über die Inhalte seiner User. Vorbild sind Bitwarden, ProtonMail, Signal.

---

4. Kryptographie-Grundlagen in Alltagssprache

Symmetrische Verschlüsselung — der Schlüsselbund-Safe

Analogie: Ein Vorhänge-Schloss. Ein Schlüssel schließt auf, derselbe Schlüssel schließt zu. Wer den Schlüssel hat, hat alles.

Vorteile: - **Sehr schnell** (Hardware-beschleunigt, gigabytes pro Sekunde) - **Sehr sicher** bei ausreichender Schlüssel-Länge (256 bit = 2^256 Möglichkeiten)

Problem: Wie bekommt der Empfänger den Schlüssel? Über das offene Internet kann man ihn nicht einfach schicken — jeder Mitlauscher bekäme ihn auch.

Lösung bei uns: Der symmetrische Schlüssel (DEK) wird entweder direkt aus einem Passwort abgeleitet (im gleichen Kopf), oder via asymmetrischem Verfahren (siehe unten) durchs Internet geschickt.

Unser Algorithmus: AES-256-GCM.

Asymmetrische Verschlüsselung — der Briefkasten

Analogie: Ein Briefkasten mit **zwei** Schlüsseln. Der **öffentliche** Schlüssel (“Public Key”) schließt nur zu — wie der Einwurf-Schlitz im Briefkasten, jeder kann reinwerfen. Der **private** Schlüssel (“Private Key”) schließt auf — nur der Besitzer hat ihn, nur er holt die Post raus.

Vorteile: - Den öffentlichen Schlüssel kann man **offen herumzeigen** — es macht nichts, wenn ihn jeder kennt. - Perfekt für Internet: Alice postet ihren Public Key, Bob verschlüsselt damit eine Nachricht, nur Alice kann sie lesen.

Nachteile: - **Sehr viel langsamer** als symmetrische Verfahren (Faktor 100–1000). - Deshalb niemals direkt für große Daten verwendet, sondern **nur** um kleine symmetrische Schlüssel auszutauschen.

Unser Algorithmus: ML-KEM-768 (für Schlüssel-Austausch) und ML-DSA-65 (für Signaturen).

Hash-Funktionen — der Fingerabdruck-Scanner

Analogie: Ein Fleischwolf, aus dem immer ein Ergebnis der gleichen Größe kommt, egal was reinkommt. Kuchenrezept rein → 64-Zeichen-Fingerabdruck raus. Elefant rein → anderer 64-Zeichen-Fingerabdruck raus. **Mini-Änderung am Input → komplett anderer Output** (Lawinen-Effekt).

Eigenschaften: - **Einbahnstraße:** Vom Fingerabdruck kommt man nie zurück zum Original. - **Deterministisch:** Gleicher Input → immer gleicher Output. - **Kollisions-resistent:** Praktisch unmöglich, zwei verschiedene Inputs zu finden, die denselben Fingerabdruck ergeben.

Wofür wir sie nutzen: - **Passwörter speichern:** Nicht das Passwort selbst, sondern sein Hash. - **Datei-Integrität:** Hash vorher + Hash nachher vergleichen → Manipulation erkennen. - **Signatur-Bausteine:** Signiert wird nie das ganze Dokument, sondern sein Hash.

Unsere Algorithmen: SHA-256 für reine Integritäts-Prüfungen, Argon2id für Passwort-Hashing (langsam gewollt!).

Key Derivation Functions (KDF) — der Passwort-Transmuter

Problem: Ein Passwort wie `HundKatze123` ist (a) viel zu kurz, (b) nicht zufällig genug, um direkt als 256-bit-Schlüssel zu dienen.

Lösung: Eine KDF nimmt das Passwort + eine Zufallszahl ("Salz") und **rechnet** daraus einen hochwertigen Schlüssel. Die KDF ist absichtlich **langsam** (ein paar Sekunden), damit ein Angreifer pro Rate-Versuch dieselbe Zeit braucht wie der legitime Benutzer. Das macht Massen-Angriffe unwirtschaftlich.

Unser Algorithmus: Argon2id mit 128 MB Speicher, 3 Iterationen, 4 Threads.

---

5. Post-Quantum — warum das nicht nur Buzzword ist

Das Problem: Shor's Algorithmus

1994 hat Peter Shor einen Algorithmus veröffentlicht, der auf einem **Quantencomputer** zwei klassische Krypto-Verfahren komplett knackt: - **RSA** (basiert auf der Schwierigkeit, große Zahlen in Primfaktoren zu zerlegen) - **ECC** (elliptische Kurven — basiert auf "diskreten Logarithmen")

Quantencomputer gibt es bereits (IBM, Google, etc.), aber sie sind noch zu klein, um echten RSA oder ECC zu knacken. **Das ändert sich vermutlich in den 2030er Jahren.**

Wichtige Konsequenz: Verschlüsselte Daten, die **heute** abgefangen und gespeichert werden, können **morgen** entschlüsselt werden, sobald Quantencomputer groß genug sind. Dieses

Szenario heißt **“Harvest-Now, Decrypt-Later”** — “heute ernten, später entschlüsseln”. Geheimdienste machen das bereits.

Die Lösung: Post-Quantum-Kryptographie

Es gibt mathematische Probleme, die **weder klassische noch Quantencomputer** effizient lösen können. Auf diesen baut man neue Verfahren:

- **Gitter-basierte Verfahren** (Lattice-based) — unser Weg. Basierend auf “Learning With Errors” (LWE).
- **Code-basierte Verfahren** — McEliece, sehr alt, sehr sicher, aber riesige Schlüssel.
- **Hash-basierte Verfahren** — nur für Signaturen, sehr langsam.
- **Multivariate Verfahren** — kleine Signaturen, aber weniger Vertrauen.

Die NIST-Standardisierung 2024

Nach **8 Jahren** weltweiter Prüfung und Wettbewerb hat NIST 2024 die ersten PQ-Standards verabschiedet:

Standard	Verfahren (alter Name)	Zweck
FIPS-203	ML-KEM (Kyber)	Schlüssel-Austausch
FIPS-204	ML-DSA (Dilithium)	Digitale Signatur
FIPS-205	SLH-DSA (SPHINCS+)	Alternative Signatur, hash-basiert

Wir nutzen **FIPS-203 und FIPS-204**, weil sie den besten Kompromiss aus Geschwindigkeit, Schlüssel-Größe und Vertrauen bieten.

Warum jetzt schon?

Weil der Migrations-Aufwand gigantisch ist. Wer **heute** PQ-native baut, hat morgen keinen Angst-Schweiß, wenn der erste 4096-Qubit-Quantencomputer verfügbar wird.

Merksatz: Klassische Symmetrische Verschlüsselung (AES) ist **nicht** von Shor bedroht (Grover schwächt sie nur um Faktor 2, deswegen nehmen wir AES-**256** statt -128). Nur **Asymmetrische** Verfahren brauchen PQ-Ersatz. Deshalb kommt PQ bei uns genau dort zum Einsatz, wo asymmetrisch gearbeitet wird: beim Schlüssel-Wrap (ML-KEM) und bei Signaturen (ML-DSA).

---

6. Die Bausteine im Detail

6.1 AES-256-GCM — der Arbeits-Panzer

Vollständig: Advanced Encryption Standard, 256-bit Schlüssel, Galois/Counter Mode.

Was macht er? - Nimmt Klartext + 256-bit Schlüssel + 96-bit Nonce → gibt Ciphertext + 128-bit Tag zurück. - **Ciphertext** = das verschlüsselte Ergebnis. - **Tag** = Echtheits-Prüfsumme. Wenn auch nur ein Bit des Ciphertext verändert wird, schlägt die Entschlüsselung fehl.

Warum "GCM"? - GCM ist eine Betriebsart ("Mode of Operation"). AES als Block-Chiffre verschlüsselt nur 128-bit-Blöcke einzeln. GCM sagt: "Verschlüssele nacheinander Block für Block **und** rechne parallel einen Integritäts-Tag." Ergebnis: Verschlüsselung und Manipulations-Erkennung in einem Durchgang. - Alternative Modi (CBC, CTR) gibt es, aber nur GCM ist AEAD-fähig (Authenticated Encryption with Associated Data).

Warum 256-bit? - 128-bit AES gilt **noch** als sicher, aber Grover's Algorithmus halbiert die effektive Sicherheit auf einem Quantencomputer. 256-bit AES → 128-bit Quanten-Sicherheit. Das reicht für die absehbare Zukunft.

Bei uns: Alle Daten-at-Rest (auf Festplatte) werden mit AES-256-GCM verschlüsselt. Der Schlüssel ist der DEK.

6.2 SHA-256 — der Standard-Fingerabdruck

Vollständig: Secure Hash Algorithm, 2nd generation, 256-bit output.

Was macht er? - Nimmt beliebig lange Daten → gibt genau 32 Byte (64 Hex-Zeichen) Fingerabdruck raus.

Beispiel:

```
Input:  "Hallo Welt"
Output: 6e4dda49 a4d5f6a4 4c73893a 6c2d0b5f 7c4a1f3a 5b8e4e9c 8d0a3f5c
2b9e1d7f
```

(Beispiel-Output, nicht echt gerechnet)

Einsatz bei uns: - **Code-Hash** für Bitcoin-Anker (das Monument). - **Integritäts-Prüfungen** von Dateien und Releases. - **Start.sh-Delta-Erkennung** (SHA1 für requirements.txt — eine schwächere Schwester, reicht für non-security-Zwecke). - **NICHT mehr** für Passwort-Hashing (dafür haben wir jetzt Argon2id).

6.3 Argon2id — der langsame Passwort-Kocher

Vollständig: Argon2 Identity-Variant (Hybrid aus Argon2i und Argon2d).

Was macht er? - Nimmt: Passwort + Salz + Parameter → gibt 32-Byte-Key raus. - Der Clou: Rechnet **absichtlich lang** (1 Sekunde) und braucht **128 MB RAM**.

Warum das wichtig ist:

Ein einziger SHA-256-Durchlauf braucht auf einer modernen GPU ~1 Nanosekunde. Eine mittelgroße Farm rechnet **1 Milliarde** Passwörter **pro Sekunde** durch. Ein 8-Zeichen-Passwort ist in wenigen Stunden durchprobiert.

Argon2id ist 1.000.000× langsamer (1 Sek statt 1 ns). Dieselbe Farm schafft **1 Passwort pro Sekunde**. Ein 8-Zeichen-Passwort braucht jetzt **31 Jahre** statt 2 Stunden. Realistisch unbezahlbar.

Zusätzlich: Jeder Versuch kostet **128 MB RAM**. GPUs haben wenig, teures RAM — das strangelt die Farm-Ökonomie zusätzlich.

Unsere Parameter (hard-coded, nicht verhandelbar):

Parameter	Wert	Begründung
Variante	Argon2id	Hybrid i+d, Standard-Empfehlung
Speicher	128 MB	Bitwarden-Niveau
Zeit-Cost	3 Iterationen	~1 Sekunde pro Versuch
Parallelität	4 Threads	moderne CPU-Auslastung
Ausgabe	32 Byte	= 256-bit Schlüssel für AES

6.4 ML-KEM-768 — der Post-Quantum-Brieftausch

Vollständig: Module-Lattice Key Encapsulation Mechanism, Parameter-Set "768".

Was macht er?

ML-KEM ist ein **Schlüssel-Einkapselungs-Mechanismus**. Statt (wie RSA) "verschlüssele beliebigen Text mit dem Public Key", macht ML-KEM ausschließlich **eins**: einen zufälligen Schlüssel erzeugen und zwischen zwei Parteien transportieren.

Drei Operationen:

```
KeyGen()     → (PublicKey, PrivateKey)
Encapsulate(PublicKey) → (SharedSecret, Ciphertext)
Decapsulate(Ciphertext, PrivateKey) → SharedSecret
```

Ablauf (Alice → Bob): 1. Bob erzeugt `(PubKey_Bob, PrivKey_Bob)`. Public veröffentlicht, Private bleibt geheim. 2. Alice ruft `Encapsulate(PubKey_Bob)` → bekommt `(SharedSecret_A, Ciphertext)`. 3. Alice schickt `Ciphertext` (öffentlich) an Bob. 4. Bob ruft `Decapsulate(Ciphertext, PrivKey_Bob)` → bekommt `SharedSecret_B`. 5. **SharedSecret_A == SharedSecret_B.** Beide haben jetzt denselben Schlüssel, ohne dass er je übers Netz lief.

Warum "768"? ML-KEM gibt's in drei Härtegraden: - ML-KEM-512: ~AES-128 Sicherheit, kleinste Keys - **ML-KEM-768**: ~AES-192 Sicherheit, **unser Standard** - ML-KEM-1024: ~AES-256 Sicherheit, größte Keys

768 ist der NIST-empfohlene Kompromiss für die meisten Anwendungen.

Schlüssel-Größen ML-KEM-768: - Public Key: 1.184 Byte - Private Key: 2.400 Byte - Ciphertext: 1.088 Byte - Shared Secret: 32 Byte

Warum PQ-sicher? Die zugrundeliegende Mathematik ist "Module Learning With Errors" (M-LWE). Grob: Man hat viele lineare Gleichungen mit **absichtlich verrauschten** Lösungen, und soll die ursprünglichen Koeffizienten finden. Für klassische und Quanten-Computer gleichermaßen schwer.

Bei uns: Der DEK wird mit ML-KEM-768 "gewrappt" (eingekapselt). Wer den Vault-Key entschlüsseln will, braucht den ML-KEM-Private-Key — der wiederum liegt AES-verschlüsselt unter der KEK.

6.5 ML-DSA-65 — die Post-Quantum-Unterschrift

Vollständig: Module-Lattice Digital Signature Algorithm, Parameter-Set "65".

Was macht er? Digitale Signaturen beweisen **Authorship + Integrität**: "Dieses Dokument wurde von mir erstellt UND seitdem nicht verändert."

Drei Operationen:

```
KeyGen()                  → (PublicKey, PrivateKey)
Sign(Message, PrivKey)    → Signature
Verify(Message, Signature, PubKey) → True/False
```

Warum "65"? Wieder drei Härtegrade: - ML-DSA-44: ~AES-128 Sicherheit - **ML-DSA-65**: ~AES-192 Sicherheit, **unser Standard** - ML-DSA-87: ~AES-256 Sicherheit

Schlüssel-Größen ML-DSA-65: - Public Key: 1.952 Byte - Private Key: 4.032 Byte - Signature: 3.309 Byte

Einsatz bei uns: - **Bitcoin-Anker**: Code-Version wird signiert und in die Bitcoin-Chain geschrieben. - **Soul-Hashes**: Bei Shidow-Agenten werden Persönlichkeits-Snapshots signiert. - **Trade-Pakete**: Export-Pakete werden signiert, Import verifiziert.

6.6 TOTP — der 30-Sekunden-Code

Vollständig: Time-based One-Time Password (RFC 6238).

Was macht er? Erzeugt alle 30 Sekunden einen neuen 6-stelligen Code, der **deterministisch** aus aktuellem Unix-Zeitstempel + geheimem TOTP-Seed berechnet wird.

Formel (vereinfacht):

```
Code = HMAC-SHA1(Seed, floor(unix_time / 30))   [auf 6 Stellen gekürzt]
```

Server und Authenticator-App (z.B. Aegis, Google Authenticator) kennen beide den Seed (bei Einrichtung über QR-Code übergeben). Beide rechnen denselben Code aus → Match = gültig.

Warum funktioniert das? - Der Seed verlässt nach der ersten Übertragung (QR-Code) **nie** das Handy des Users. - Angreifer sieht nur 6-stellige Codes, die 30 Sekunden gültig sind. Klauen und Wiederverwenden? Nutzlos nach 30 Sek.

Unser Label-Format:

```
Issuer  = "Kneipe" / "ShinNexus" / "ShinPing" / "Shidow"
Account = Username (z.B. "hasi")
```

Im Authenticator sichtbar als:

```
┌──────────────────┐
│ Kneipe           │
│ hasi             │
│ 123 456          │
└──────────────────┘
```

---

7. Die Schlüssel-Kaskade (KEK → DEK → Daten)

Der Kern der Schicht-1-Architektur. Dreistufig, jede Stufe hat eine klare Aufgabe.

Die drei Ebenen

```
   ┌──────────────────┐
   │  VAULT-DATEN     │  ← Identität, Users, Einstellungen, Profile
   │  (AES-256-GCM)   │
   └────────▲─────────┘
            │ entschlüsselt mit
   ┌────────┴─────────┐
   │      DEK         │  ← Data Encryption Key (256-bit, einmalig erzeugt,
unveränderlich)
   │  (ML-KEM-Wrap)   │
   └────────▲─────────┘
            │ entwrappt mit
   ┌────────┴─────────┐
   │ ML-KEM-PrivKey   │  ← Privater PQ-Schlüssel
   │  (AES-256-GCM)   │
   └────────▲─────────┘
            │ entschlüsselt mit
   ┌────────┴─────────┐
   │      KEK         │  ← Key Encryption Key (nur aus PW + Salz + machine-id
ableitbar)
   │   (Argon2id)     │
   └────────▲─────────┘
            │ abgeleitet aus
   ┌────────┴─────────┐
   │ Owner-Passwort   │  ← im Kopf des Owners
   │    + machine-id  │  ← im Betriebssystem
   │    + .salt       │  ← Datei im Vault-Ordner
   └──────────────────┘
```

Die einzelnen Dateien

Im Vault-Ordner liegen:

Datei	Inhalt	Schlüssel
`.salt`	16 Byte Zufall (unverschlüsselt!)	— (ist ein Salz, kein Secret)
`kem_priv.vault`	ML-KEM-Private-Key	AES-GCM(KEK)
`kem_priv.seed.vault`	ML-KEM-Private-Key, zweite Kopie	AES-GCM(Seed-Key) — Recovery-Pfad
`dek.wrap.vault`	DEK, eingekapselt	ML-KEM-768 Encapsulation
`identity.vault`	Owner-Identität	AES-GCM(DEK)
`users.vault`	User-Datenbank	AES-GCM(DEK)
(weitere)	...	AES-GCM(DEK)

Der Entschlüsselungs-Ablauf (beim Owner-Login)

Einmal beim Login: 1. Owner gibt Passwort ein. 2. Server liest `.salt` und liest `machine-id` vom Betriebssystem. 3. **KEK ableiten:** `KEK = Argon2id(password + .salt + machine-id)` — dauert ~1 Sekunde. 4. **ML-KEM-Private entschlüsseln:** `privkey = AES-GCM-Decrypt(kem_priv.vault, KEK)`. 5. **DEK entwrappen:** `DEK = ML-KEM-Decapsulate(dek.wrap.vault, privkey)`. 6. **DEK bleibt im RAM**, solange der Server läuft.

Danach für jede Vault-Datei: - `klartext = AES-GCM-Decrypt(datei.vault, DEK)`

Schritt 3–5 laufen **einmal**. Ab dann ist Step 6 super-schnell (Gigabytes pro Sekunde).

Warum die Trennung KEK ↔ DEK?

Der alte Ansatz (vor 2026-04-16): Ein einziger Master-Key aus `SHA256(PW + machine-id)`, direkt zum Verschlüsseln aller Dateien verwendet.

Zwei Probleme: 1. **Owner-Passwort-Änderung = Katastrophen-Risiko.** Alle Vault-Dateien müssen neu verschlüsselt werden. Fällt der Strom mitten im Re-Encrypt aus → halb alter, halb neuer Vault → **kaputt**, nicht wiederherstellbar. 2. **Nicht PQ-nativ.** Nirgendwo kam asymmetrische Crypto vor. Der Anspruch "Monument-Grade" war leere Marketing-Hülle.

Der neue Ansatz mit KEK/DEK löst beides:

1. **PW-Change = atomisch.** Nur **eine** Datei (`kem_priv.vault`) wird neu verschlüsselt (mit neuer KEK). Der DEK bleibt gleich, Vault-Dateien unberührt. Wenn Strom mittendrin ausfällt: alter `kem_priv.vault` noch da → nichts verloren.
2. **PQ-nativ.** Der Zwischen-Layer ML-KEM-768 ist der einzige asymmetrische Wrap, und er ist nach NIST-FIPS-203 offiziell post-quantum-sicher.

Owner-Passwort-Wechsel — atomar und risikofrei

```
1. KEK_alt = Argon2id(PW_alt + .salt + machine-id)
2. privkey = AES-GCM-Decrypt(kem_priv.vault, KEK_alt)
3. KEK_neu = Argon2id(PW_neu + .salt + machine-id)
4. AES-GCM-Encrypt(privkey, KEK_neu) → kem_priv.vault (überschreibt)
5. Fertig.
```

Bei Crash nach Schritt 1–3: Nichts verloren (alter `kem_priv.vault` steht noch). Bei Crash in Schritt 4: Die Datei ist entweder komplett alt oder komplett neu — das Dateisystem (ext4, ZFS, NTFS) garantiert atomische Schreibvorgänge für Einzel-Dateien.

---

8. Der Salzstreuer — was er macht und warum

Das Problem: Rainbow-Tables und Leak-Langzeit-Wirkung

Angenommen, ein Angreifer klaut den Vault-Ordner (inkl. `.salt`). Er kann offline rechnen und das Passwort erraten. Argon2id macht das zwar sehr langsam, aber theoretisch möglich bei schwachen Passwörtern oder gigantischen Rechenkapazitäten (Staatsakteure).

Wenn er das Passwort **2 Jahre nach dem Leak** noch errät, ist der Vault aus der damaligen Zeit immer noch entschlüsselbar.

Die Lösung: Salz-Rotation (der Salzstreuer)

Ein Button im Sicherheits-Tab der UI, der das `.salt` **neu würfelt**. Mechanisch dieselbe Operation wie PW-Wechsel:

1. Owner klickt den Salzstreuer.
2. Abfrage: aktuelles Passwort + aktuelles TOTP (2FA-Pflicht).
3. `KEK_alt = Argon2id(PW + .salt_alt + machine-id)` → `kem_priv.vault` öffnen → `privkey` im RAM.
4. **Neues Salz würfeln** (16 Byte Zufall).
5. `KEK_neu = Argon2id(PW + .salt_neu + machine-id)`.
6. `AES-GCM-Encrypt(privkey, KEK_neu)` → `kem_priv.vault` überschreiben.
7. `.salt` mit `salt_neu` überschreiben.
8. **Fertig.** DEK unberührt. Vault-Dateien unberührt.

Warum das wirkt

Der Angreifer hat den Vault-Zustand **vom Zeitpunkt X**. Nach dem Salzstreuer hat er immer noch denselben Zustand — aber der Vault auf dem Server ist jetzt ein **anderer** (mit neuem Salz und neu-gewrapptem Private Key). Wenn der Angreifer das Passwort in 2 Jahren errät, entschlüsselt er nur den Stand von vor 2 Jahren. Alle Änderungen seitdem sind in einem **anderen Krypto-Kontext** und für ihn wertlos (solange die neuen Vault-Dateien nicht auch geleakt wurden).

Wichtige Regel: 24-Stunden-Cooldown

Der Salzstreuer darf **maximal 1× pro 24 Stunden** ausgelöst werden. Grund: Jede Rotation lädt 128 MB RAM und kostet ~1 Sek CPU. Spam-Schutz gegen Klick-Wütige oder automatisierte Scripts.

Innerhalb der 24 h antwortet der Server mit einer programmspezifischen Toast-Meldung:

- **Kneipe:** *"Ruhig, Kompaniechef. Mehr Salz gibt's erst mit dem nächsten Tequila — morgen."*
- **ShinNexus:** *"Das Salz ist heilig, Padawan. Verwende es sparsam — morgen wieder."*
- **ShinPing:** *"Zu viel Salz macht krank. Das Körnchen von heute muss reichen."*
- **Shidow:** *"Der Schatten ist noch salzig genug. Operation ausgesetzt — bis morgen."*

Wichtig: Salz ist **nicht** geheim

Im `.salt` stehen 16 Byte Zufall. Die Datei ist **unverschlüsselt**. Das ist korrekt so. Ein Salz schützt nicht durch Geheimhaltung, sondern durch **Einzigartigkeit** — es verhindert, dass ein Angreifer eine vorberechnete Passwort-Tabelle ("Rainbow Table") für Milliarden Accounts gleichzeitig nutzt. Er muss für **jedes Salz einzeln** alle Rate-Versuche neu rechnen.

Konsequenz: Die `.salt`-Datei MUSS ins Backup. Ohne sie ist der Vault auch mit korrektem Passwort nicht mehr entschlüsselbar — die KEK-Ableitung bräuchte exakt dieses Salz.

---

9. Zwei-Faktor-Authentifizierung (2FA)

Das Prinzip

Ein Faktor reicht nicht. Passwörter werden gephisht, gestohlen, Brute-Force-geraten. Deshalb: **zwei** unabhängige Beweise der Identität:

Faktor-Typ	Beispiel	Problem allein
Etwas, das du weißt	Passwort, PIN	Phishbar, teilbar, ratbar
Etwas, das du hast	Handy, Hardware-Token	Klaubar, verlierbar
Etwas, das du bist	Fingerabdruck, Gesicht	Biometrie kann man nicht ändern wenn kompromittiert

2FA kombiniert mindestens zwei der drei. Bei uns: **Passwort** (wissen) + **TOTP aus Authenticator-App** (haben = Handy).

Einrichtung

1. User klickt "2FA aktivieren".
2. Server erzeugt 20 Byte Zufalls-Seed (`TOTP_SECRET`).
3. Server zeigt QR-Code: `otpauth://totp/Kneipe:hasi?secret=BASE32-SEED&issuer=Kneipe` .
4. User scannt QR mit Authenticator-App (Aegis, Authy, Google Authenticator, ...).
5. App speichert den Seed dauerhaft auf dem Handy.
6. User tippt zum Beweis einmal einen aktuellen Code ein.
7. Server verifiziert → 2FA ist aktiv, der Seed wird verschlüsselt in der User-DB abgelegt.

Jeder Login

1. User gibt Passwort + aktuellen 6-stelligen TOTP-Code ein.
2. Server prüft Passwort-Hash.
3. Server holt den **verschlüsselten TOTP-Seed** aus der User-DB, entschlüsselt ihn (mit DEK aus Schicht 1), berechnet den aktuellen Code selbst.

4. Vergleich: User-Code == Server-Code → **eingeloggt**.
5. Kleine Toleranz: ±30 Sek (Uhren-Drift zwischen Handy und Server).

Warum auch der Seed verschlüsselt sein muss

Wenn der Server gehackt wird und der Angreifer den Seed im Klartext findet → er kann auf seinem eigenen Gerät alle zukünftigen Codes rechnen → 2FA wertlos. Deshalb: Seed ist in `users.vault` abgelegt, nur entschlüsselbar mit DEK → nur verfügbar, wenn Schicht 1 offen ist (Server läuft + Owner hat entsperrt).

---

10. Die drei Sicherheits-Schichten

Die Architektur hat **drei übereinanderliegende Schalen**. Jede löst ein anderes Problem.

Schicht 1: Der Server-Master (Kapitänskajüte)

Was sie schützt: Alles was auf der Festplatte liegt. **Schlüssel:** KEK-Kaskade (Owner-Passwort + .salt + machine-id → KEK → Private-Key → DEK → Daten). **Wer hat Zugriff:** Nur der Owner, und nur während der Server läuft und der Owner eingeloggt war.

Auswirkung: - Server aus → alles unlesbar (DEK war nur im RAM, jetzt weg). - Festplatte geklaut → unlesbar. - Dateien auf andere Maschine kopiert → unlesbar (machine-id fehlt / falsch). - Server an, Owner eingeloggt → Server kann normal arbeiten.

Schicht 2: Die User-Schließfächer (Mannschafts-Decks)

Was sie schützt: Persönliche Daten jedes einzelnen Users (Privatnachrichten-History, persönliche Einstellungen, eigene Dateien). **Schlüssel:** User-Datenschlüssel, aus User-Passwort im Browser abgeleitet. **Wer hat Zugriff:** Nur der User selbst.

Wichtig: - Das User-Passwort geht **nie** roh zum Server. Der Server hat nur einen Hash davon (Argon2id). - Der **Entschlüsselungs-Schlüssel** wird im Browser berechnet: `userkey = Argon2id(password + salt + user_id)`. Verlässt den Browser nie. - Der Server liefert dem Browser **verschlüsselte Blobs** aus der DB. Der Browser entschlüsselt lokal. - **Folge:** Selbst der Owner (der Schicht 1 offen hat!) kann die User-Blobs nicht lesen. Er sieht nur Ciphertext.

Der Session-Token (nicht verwechseln mit dem Datenschlüssel!): - Bekommt der User nach erfolgreichem Login vom Server. - Dient als **Zugangs-Ausweis** (`Authorization: Bearer <token>`-Header bei jeder API-Anfrage). - Läuft nach Idle-Zeit ab. - Entschlüsselt **nichts** — ist nur eine Zutritts-Marke.

Schicht 3: Gruppen-Schlüssel (Tische, Räume, Channels)

Was sie schützt: Nachrichten in gemeinsamen Räumen (Kneipen-Tische, ShinShare-Chats, Shidow-Hive-Rooms). **Schlüssel:** Gruppen-Schlüssel, einmalig erzeugt beim Spawn des Raums. **Wer hat Zugriff:** Alle Mitglieder des Raums.

Verteilung: 1. User tritt einem Raum bei. 2. Ein bestehendes Mitglied (oder der Server im Auftrag des Ersten) nimmt den Raum-Schlüssel und verschlüsselt ihn mit dem **Public Key** des neuen Users (ML-KEM-768). 3. Der verschlüsselte Schlüssel wird dem neuen User geliefert. 4. Er entschlüsselt ihn mit seinem Private Key → er hat jetzt den Raum-Schlüssel.

Nachrichten-Flow: 1. User A schreibt Nachricht im Browser. 2. Browser verschlüsselt: `AES-GCM(message, raum_key)` → Ciphertext. 3. Ciphertext geht an Server. 4. Server verteilt an alle Raum-Mitglieder (polling). 5. Jeder Mitglieds-Browser entschlüsselt mit seinem Raum-Schlüssel lokal.

Server sah nur Ciphertext. Er kann mitschreiben wer wann mit wem war (Metadaten), aber nicht **was** gesagt wurde.

---

11. Recovery: Seed-Phrase und Passwort-Vergessen

Das Dilemma

Echtes Zero-Knowledge heißt: **Der Server kann Passwörter nicht wiederherstellen.** Wenn er sie wiederherstellen könnte, hätte er eine Hintertür — die ein Angreifer auch ausnutzen könnte.

Aber: Passwörter werden vergessen. Das Leben ist kein Security-Audit.

Die Lösung: Seed-Phrase

Bei Registrierung bekommt jeder User eine **12- oder 24-Wort-Phrase** angezeigt (aus BIP-39 Wörterliste, wie bei Bitcoin-Wallets). Diese Phrase ist **deterministisch** mit seinem Datenschlüssel verknüpft:

```
seed_phrase → BIP-39 → seed_bytes → Argon2id → parallel_userkey
```

Der User muss diese Phrase **ausdrucken, aufschreiben, sicher verwahren**. Sie ist sein **Notfall-Schlüssel**.

Zusätzliche Recovery-Datei (zweiter Key-Wrap)

Parallel zu `kem_priv.vault` wird eine zweite Kopie angelegt:

- `kem_priv.vault` — verschlüsselt mit KEK (= aus Passwort abgeleitet)
- `kem_priv.seed.vault` — verschlüsselt mit Seed-Key (= aus Seed-Phrase abgeleitet)

Beide entschlüsseln denselben ML-KEM-Private-Key → denselben DEK → dieselben Vault-Daten. Zwei unabhängige Zugangs-Wege.

Der Recovery-Flow

1. User klickt "Passwort vergessen".
2. Server sendet Bestätigungs-Email: *"Bist du sicher? **7 Tage Gnadenfrist**, danach Account weg wenn du nicht mit Seed kommst."*
3. **Zwei Optionen:**
   - **Option A:** User findet altes Passwort doch noch (Browser-Cache, Notizbuch) → normaler Login → Reset-Prozess wird abgebrochen.
   - **Option B:** User gibt Seed-Phrase ein → Server leitet Seed-Key ab → entschlüsselt `kem_priv.seed.vault` → hat Private Key → kann DEK entwrappen → alle Vault-Daten noch lesbar. User setzt neues Passwort → neuer KEK → `kem_priv.vault` wird neu geschrieben.
4. Nach 7 Tagen ohne Login und ohne Seed: **Account-Löschung.** Email wieder frei für Neu-Registrierung.

Zeit-Schlüsselwort: 7 Tage

- **Kurz genug**, dass ein geklauter Account nicht wochenlang als Phishing-Vehikel dient.
- **Lang genug**, dass der legitime Besitzer nachdenken, den Notizblock finden, die Familie fragen kann.

Keine Hintertür durch den Owner

Selbst wenn der Owner des Servers die Daten wiederherstellen wollte (weil er der User bittet) → er hat keinen Weg. Er hat kein User-Passwort, keine Seed-Phrase. Die User-Vault-Blobs sind für ihn Müll. **Das ist Feature, nicht Bug.**

---

12. Was Shinpai-AI **nicht** tut — die Eisernen Regeln

Diese Regeln wurden schmerzhaft erlernt und sind **nicht verhandelbar**. Jede Verletzung macht das ganze Konzept zu Theater.

Regel 1: Kein Fallback-Pfad

"Fallback bei Security = Megafail."

Wenn ein Auth-Mechanismus abgeschaltet ist (weil ein stärkerer da ist), darf **kein Fallback zum alten** eingebaut werden. Das öffnet eine Hintertür.

Beispiele für verbotene Muster: - “Wenn Argon2id zu langsam ist, fallback auf SHA-256” — **Nein.** Argon2id bleibt, ohne Ausnahme. - “Wenn Bearer-Token fehlt, versuche Cookie-Auth” — **Nein.** Wenn Bearer abgeschaltet wurde, ist das der neue Standard. Kein Retro-Weg. - “Migration alter Vaults bei Versions-Wechsel” — **Nein.** Alte Vaults werden manuell neu aufgesetzt, nicht still migriert.

Regel 2: Kein Auto-Unlock ohne Owner-Login nach Reboot

Der **Igni** (Zündschlüssel-Datei) macht Bequemlichkeit möglich: nach Reboot kein Owner-Login nötig, Server startet durch. **Aber:** Wenn die Igni-Datei gelöscht wird oder der Server auf eine andere Maschine umzieht (= andere machine-id), **muss** der Owner sich einloggen. Es darf keinen zweiten Weg geben, den Master-Key zu entsperren — sonst wäre das der Hauptangriffsvektor.

Regel 3: Parameter sind hardcoded

Argon2-Parameter (128 MB / 3 / 4 / 32) sind **im Code festgenagelt**, nicht via Config einstellbar. Grund: Ein “schwacher Modus für schwache Hardware” wäre genau die Sicherheitslücke, die der Angreifer sucht. Wenn die Hardware Argon2id nicht packt, ist die Hardware falsch, nicht der Algorithmus.

Regel 4: `.salt` ist Pflicht im Backup

Die Salz-Datei ist der letzte Puzzle-Stein der KEK-Ableitung. Fehlt sie → Vault tot, auch mit korrektem Passwort. Sie **muss** ins Backup, und sie **muss** aus dem Vault-Ordner kommen (nicht aus dem Igni verschoben werden).

Regel 5: Salzstreuer nur bei aktivem 2FA

Der Salzstreuer-Button ist **unsichtbar**, wenn 2FA nicht eingerichtet ist. Grund: Er ist ein Security-sensitives API-Gate, und sensitive Gates ohne zweiten Faktor sind eine Einladung.

Regel 6: Owner sieht keine User-Inhalte

Selbst wenn der Owner Schicht 1 offen hat, kommt er an **keine** User-Inhalte. Das ist die Zero-Knowledge-Garantie. Sie gilt auch, wenn: - Der Owner “nur kurz debuggen will” - Behörden eine Anfrage stellen - Der User selbst den Owner darum bittet

Der einzige Weg zu User-Inhalten führt über den User selbst + sein Passwort oder seine Seed-Phrase. Punkt.

Regel 7: Keine Krypto-Eigenerfindungen

Alle Algorithmen sind **NIST-standardisiert oder seit Jahrzehnten peer-reviewed**:

Zweck	Algorithmus	Standard
Symmetrische Verschlüsselung	AES-256-GCM	FIPS-197 + NIST-SP-800-38D
Hash	SHA-256	FIPS-180-4
Passwort-KDF	Argon2id	RFC 9106 (IRTF)
PQ Key-Wrap	ML-KEM-768	FIPS-203
PQ Signatur	ML-DSA-65	FIPS-204
2FA	TOTP	RFC 6238

Nie eigene Algorithmen, "optimierte" Varianten, "schnellere Versionen". Der einzige sichere Weg ist der ausgetretene.

Regel 8: Wenn jemand drängt — ablehnen

"Mach das mal schnell, ohne Argon2id", *"Wir brauchen eine Backdoor für Support-Fälle"*, *"Gib mir nur das User-Passwort zum Testen"*. **Nein, nein, nein.** Sicherheit kennt kein "nur-mal-schnell". Jede dieser Bitten ist entweder (a) legitim aber dann anders lösbar, oder (b) ein Social-Engineering-Versuch.

Anhang: Wo welche Komponente im Code lebt

Komponente	Datei	Was
PQ-Schlüsselpaar erzeugen	`ShinNexus.py _ensure_keypair()`	ML-DSA-65 + ML-KEM-768 via `oqs`-Bibliothek
Vault-Unlock	`ShinNexus.py vault_unlock()`	KEK-Ableitung + Kaskade
Symmetrische Verschlüsselung	`ShinNexus.py vault_encrypt/decrypt`	AES-256-GCM
TOTP Verify	alle Programme (`pyotp`)	RFC 6238
Login-Endpunkte	`ShinNexus.py /api/auth/login`, `Kneipe server.py`	PW-Hash-Check + Token-Issue

Die konkreten Python-Bibliotheken: - **`argon2-cffi`** — Argon2id Bindings (C-Referenz-Impl, nicht reines Python!) - **`cryptography`** — AES-GCM, SHA-256, HMAC (OpenSSL-backed) - **`oqs`** (liboqs-python) — ML-KEM, ML-DSA (NIST PQ-Standards, Open-Source-Impl) - **`pyotp`** — TOTP/HOTP

Alle vier sind auf allen drei Zielplattformen (Linux, Windows, Android) installiert und getestet. Die Windows-Build-Pipeline baut `liboqs` aus C-Source selbst, weil der `pip install`-Pfad auf Windows kaputt ist.

Erstellt: 2026-04-22 Basis: `PQ-Architektur.md` — Shinpai-AI Ökosystem Ziel dieser Doku: Jeder Begriff verstehbar, jede Abkürzung erklärt, jede Entscheidung nachvollziehbar.